RESEARCH AND VERIFICATION NOTE

Proof Transport for the 215-State ZFC Machines

A local CM215 certificate, a source-proof audit, and bounded searches for 214 states

Prepared for the ProveIt repository
21 October 2026

Abstract

The most useful simplification for a second verified 215-state machine is to transfer correctness directly from the already calibrated K215 table. The existing conditional-marker candidate CM215 changes six transition rows and keeps the source program unchanged. We replace its history-based explanation by a three-clause inductive tape invariant. Each old transition then has an exact one- or three-step implementation, preserving the entire tape and head at the comparison points. A reusable positive-block theorem turns this single simulation into halting equivalence in both directions.

The accompanying artifacts include the generic theorem checked in Lean 4.32.0 and Coq 8.18.0, the concrete Lean CM215 development, literal machine tables, finite checks, and an interface to the repository's dense-machine semantics. They separate new checks from the retained K215 calibration. The current repository already incorporates three proof-simplification phases; this report credits those changes and identifies the remaining source-certificate opportunity: 2,240 zero-offset checkpoints use only 32 local arithmetic-effect signatures, and their labelled graph has a checked 1,423-class quotient.

No verified 214-state machine was obtained. Completed searches evaluate 17,708,928 fold specifications across fourteen literal tables and two initial domains, plus 159,509,360 individual branch specifications in a second restricted family. None survives the stated finite-window filters. These are reproducible search boundaries, not state lower bounds.

Pinned repository snapshot

`b6c7087477e51c017003bd729e712c8969733223`

This contribution concerns proof reuse and verified local transformation. The 215-state tables themselves and their foundational calibration are inherited from ProveIt and its credited upstream authors.

Contents

1 Answers and evidence boundaries **2**
1.1 The target statement . . . 2
1.2 Machine model and identities . . . 2

2 What the current proofs already achieve **3**
2.1 The endpoint facade is not the expensive part . . . 3
2.2 Proof reuse should follow the kind of change . . . 4

3 A reusable positive-block theorem **4**
3.1 Why positivity matters . . . 5
3.2 Checked implementations . . . 5

4 CM215: a three-clause invariant **5**
4.1 Exact local blocks . . . 6
4.2 The concrete finite evidence . . . 7
4.3 Connection to the existing endpoint . . . 7

5 Which 215-state candidate is easier to verify? **8**

6 A concrete next simplification of the source proof **9**
6.1 Remove unused offset generality carefully . . . 9
6.2 Separate local effects from addresses . . . 9
6.3 What this measurement does and does not buy . . . 10

7 Completed searches for a 214-state machine **10**
7.1 Pinned inputs and abstract contexts . . . 10
7.2 Family A: inherited rows and one common branch . . . 10
7.3 Family B: independently synthesized branches . . . 11
7.4 Controls and interpretation . . . 11

8 Reproduction and integration **12**

9 Further research questions **12**

Attribution **13**

1 Answers and evidence boundaries

The recommended next step is to retain the checked K215 calibration as a base theorem and attach small machine-to-machine proofs for local variants. For CM215 this removes the need to redo the source enumerator, numerical calculus, register compiler, 4,751 primitive-source occurrences, and blank initialization. It reduces the *additional proof* needed for another candidate. The base mathematics remains in the final dependency chain.

The question about fewer states has a different outcome: this work does not supply a correct 214-state machine. It supplies complete, bounded, reproducible searches and a reusable way to certify a future successful local transformation.

Result	Status and scope
Exact K215 consistency endpoint	Existing repository theorem and retained Lean/Rocq build evidence; the complete baseline was not rebuilt here.
Positive-block transport	New small checked artifact in Lean 4.32.0 and Coq 8.18.0; the Coq object also passed recursive `coqchk`.
CM215 local transformation	Three-clause invariant, exact whole-tape blocks, and a concrete Lean transfer development. The literal candidate already existed in the repository.
Source-certificate analysis	Fresh finite computation: 2,240 entries, zero nonzero offsets, 32 effect signatures, 1,423 quotient classes. This is not yet a replacement kernel checker.
Search below 215 states	Two precisely limited families completed; no 214-state candidate obtained and no minimality claim.

1.1 The target statement

Let $\mathrm{Ref}_{\mathrm{ZFC}}$ mean that there is a finite syntactic derivation of falsity in the repository's ZFC presentation, and let $\mathrm{Con(ZFC)} \equiv \neg\,\mathrm{Ref}_{\mathrm{ZFC}}$. The base endpoint states

$$\mathrm{Halts}(\mathrm{K215}, \mathrm{blank}) \iff \mathrm{Ref}_{\mathrm{ZFC}}\,.$$

No consistency hypothesis is a premise of this equivalence. Combining it with a proof that K215 and CM215 halt equivalently gives

$$\neg\,\mathrm{Halts}(\mathrm{CM215}, \mathrm{blank}) \iff \mathrm{Con(ZFC)}.$$

The repository records a distinction between an external proof-assistant theorem and a derivation encoded in ZFC's own proof syntax. We preserve it. The new finite-machine argument can be formalized in the same elementary arithmetic setting as the original machine argument; this packet does not claim to have constructed a new encoded internal ZFC or PA derivation. The existing ordinary ZFC-provability conclusion transports by elementary propositional reasoning. See Section 4.3.

1.2 Machine model and identities

All machine counts use one two-way infinite tape, two symbols, blank symbol zero, and only left/right moves. Initialization is part of the working-state count; the distinct halt is excluded. Both dense

tables have working states $0, \ldots, 214$, initial control zero, and halt 215. Their native tables preserve historical labels and use halt 254.

The dense K215 table has SHA-256

```
fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090
```

and the dense CM215 table has SHA-256

```
1f4f084f7293e2b042d5746039f793ff72ed429083b504970c939a5515e666cf
```

Both files are included, with 430 rows each. The root manifest binds the literal data, new proof sources, and all retained evidence. Machine hashes identify the theorem's target; a same-size table is not interchangeable with a calibrated one.

2 What the current proofs already achieve

The pinned snapshot already contains three simplification phases. The verification index records an original fresh Lean build of 204 project modules and two explicit clients, and a Rocq build of 237 selected sources with recursive checking. The current Lean closure is 185 project modules. The phase-three record rebuilt nine affected modules and reused 176 hash-bound objects; it is an incremental check, not a second full replay. The final Lean reports use only `propext`, `Classical.choice`, and `Quot.sound`; the four retained Rocq endpoint dependency reports are closed under the global context.[1, 2]

Proof source	Original bytes	Current bytes
Lean dominated implication	689,857	35,185
Lean AX routine	202,222	7,563
Lean source-certificate data	653,569	456,548
Lean site-coverage facade	107,243	14,916
Lean register determinism	17,209	12,988
Rocq dominated implication	921,691	22,673

These are repository-recorded source-size reductions, not performance improvements measured in this session. In particular, symmetry already removed the need for sixteen reverse source-certificate check modules. The latest Lean dominated-rule development already uses 61 bootstrap lemmas, 81 conditional schemas, and a deduction theorem. The existing enumerator proof already shares one theorem between the zero and boot starts. The site facade already directly composes 105 checked blocks. Recommending these as new work would duplicate completed work.

2.1 The endpoint facade is not the expensive part

The Lean endpoint file is 53 lines. The corresponding Rocq sparse and dense endpoint facades are 19 and 18 lines. The Lean facade supplies the following machine-specific inputs to a shared theorem:

1. A commuting dense-state renaming into the sparse machine table.
2. Agreement with already proved register-engine contracts.

3. A blank bootstrap reaching the correct dispatcher frame.
4. Coverage of every literal source occurrence.
5. Source/program correspondence and the numerical-calculus refutation equivalence.

The final composition is already appropriately small. The profitable targets are generated certificates, repetitive local contracts, and repeated verification of closely related candidates.[3]

2.2 Proof reuse should follow the kind of change

There are three different situations. A local table rewrite with unchanged source is best handled by direct machine simulation. A source reassociation with the same command semantics is best handled by a source control-flow certificate plus existing engine contracts. A new command language or proof calculus needs a semantic calibration at that higher level before the compiler can be reused.

This separation prevents a small physical edit from triggering a new proof of set theory. It also prevents an apparently shorter source from being accepted merely because its table and startup look plausible. The proof interface must expose every changed semantic assumption.

3 A reusable positive-block theorem

Let $f : A \to A \cup \{\bot\}$ and $g : B \to B \cup \{\bot\}$ be deterministic partial transition functions. The value $\bot$ means that no transition exists. Write $f^n(a) \downarrow a'$ for a successful n-step execution; this includes the assertion that no earlier step became undefined. Define

$$\mathrm{Halts}_f(a) \quad \Longleftrightarrow \quad \exists n, a' \; (f^n(a) \downarrow a' \;\wedge\; f(a') = \bot).$$

Theorem 3.1 (Positive-block transport)**.** *Suppose a relation $R \subseteq A \times B$ has the following properties whenever $a \, R \, b$:*

1. *$f(a) = \bot$ if and only if $g(b) = \bot$.*
2. *If $f(a) = a'$, then for some $k > 0$ and b', $g^k(b) \downarrow b'$ and $a' \, R \, b'$.*

Then $a \, R \, b$ implies $\mathrm{Halts}_f(a) \leftrightarrow \mathrm{Halts}_g(b)$*. One forward simulation suffices; no reverse certificate is needed.*

Proof. For preservation of halting, induct on the number of source steps before the terminal configuration. At zero steps use terminal agreement. At a positive step, the second condition provides a successful target block. Concatenate that block with the terminating target execution provided by the induction hypothesis at its endpoint.

For reflection, fix a terminating target run of length n and use strong induction on n. If the source is terminal, it already halts. Otherwise the second condition supplies a successful target prefix of length $k > 0$. Determinism forces it to agree with the corresponding initial part of the chosen terminating run. It cannot pass the terminal endpoint, because a terminal configuration has no successor. Thus $n = k + m$ for some $m < n$, and the target terminates from b' after m further steps. Apply the induction hypothesis to $a' \, R \, b'$ and then prepend the one source step. □

3.1 Why positivity matters

If zero-length implementations were allowed, let the source loop at a forever, and let the target take one transition from b to a terminal state. Relate only a and b. Terminal agreement holds at the related pair, and every source step is represented by zero target steps, yet only the target halts. A well-founded auxiliary ranking could replace strict positivity in a more general theorem; it cannot simply be omitted.

There is no fairness assumption in Theorem 3.1. The result concerns deterministic partial systems, and successful positive prefixes provide exactly the progress required by the reverse induction. There is also no uniform upper bound on k in the theorem. CM215 happens to use only $k = 1$ and $k = 3$.

3.2 Checked implementations

`formal/PositiveStutter.lean` imports only `Init` and implements the theorem for arbitrary configuration types. Its halting and nonhalting equivalences were compiled with Lean 4.32.0. Their printed axiom dependencies are `propext` and `Quot.sound`, with no `sorryAx`.

`formal/PositiveStutter.v` gives the same theorem in the Coq/Rocq language. It was compiled here with *Coq 8.18.0*, and both public equivalences print "Closed under the global context." Its compiled object passed `coqchk`. The pinned repository uses Rocq 9.1.0 for its full baseline; this session's generic Coq check is stated with its actual version and is not represented as a new full Rocq endpoint build.

The theorem is a standard simulation principle. The contribution is a small reusable implementation, its application to the literal CM215 transformation, and the exact separation of local obligations from the large existing calibration.

4 CM215: a three-clause invariant

CM215 is the existing conditional-marker alternative. It shares the source program with K215, including its 202 distinct source subtrees and 4,751 primitive occurrences. It changes six native transition rows. Its original packet supplies an ordinary whole-tape simulation theorem and separate runtime evidence.[4]

Native state/read	K215 action	CM215 action
52/0	0 R 154	0 L 176
156/1	1 R 176	1 R 52
165/0	0 R 176	0 R 52
176/0	0 R 168	0 R 185
176/1	1 R 150	1 R 129
193/0	0 R 176	0 R 52

CM215 merges the old operational roles of 52 and 176 into control 52, then reuses physical control 176 as a selector. This is a state-neutral transformation: it leaves 215 working states. It does not itself save the additional state sought by the user.

For a configuration (q, h, t), let $t : \mathbb{Z} \to \{0, 1\}$ be the entire tape. There is no finite-support restriction in the local theorem.

Definition 4.1. The invariant $I(q, h, t)$ is the conjunction

$$\begin{aligned} q = 52 &\implies t(h-1) = 1, \\ q = 176 &\implies \big(t(h) = 1 \ \vee \ t(h-1) = 0\big), \\ q = 156 &\implies t(h+1) = 1. \end{aligned}$$

These three clauses replace a history-sensitive explanation by a one-step invariant. The third clause records exactly the information needed for the exceptional incoming path into 176; no unbounded history or auxiliary trace is retained.

Lemma 4.2 (Invariant preservation)**.** *Every K215 execution starting in control zero satisfies I, for any initial head position and any initial binary tape.*

Proof. At control zero all three implications are vacuous. For preservation, inspect the complete incoming rows of the three distinguished controls. Control 52 has incoming rows 190/1 and 193/1, each writing one and moving right. Hence the new cell immediately to the left is one.

Control 156 has the unique incoming row 189/1, which writes one and moves left. Hence the new cell immediately to the right is one.

Control 176 has incoming rows 165/0, 193/0, and 156/1. The first two write zero and move right, establishing the left-zero disjunct. The last moves right from 156. The induction hypothesis for 156 says that the cell it is about to read is one, establishing the current-one disjunct. No other destination can make one of the implications nonvacuous. Induction on the execution length completes the proof. □

4.1 Exact local blocks

Define $\phi(176) = 52$ and $\phi(q) = q$ otherwise. At a comparison point relate (q, h, t) to $(\phi(q), h, t)$, with $I(q, h, t)$ required on the old configuration.

For every old control outside $\{52, 176\}$, the new row is the old row with its target relabelled by ϕ. Thus the corresponding one-step executions agree on the whole tape and head. At both exceptional old controls the read-one row is $1\,R\,150$, and the new control 52 has that same row. These cases also take one step.

When old control 52 reads zero, the invariant gives $t(h-1) = 1$, and the new computation is

$$52 \text{ at } h \xrightarrow{0L} 176 \text{ at } h-1 \xrightarrow{1R} 129 \text{ at } h \xrightarrow{0R} 154 \text{ at } h+1.$$

Each written bit equals the bit already present. This is exactly the head, tape, and destination of the old 52/0 step.

When old control 176 reads zero, its invariant forces $t(h-1) = 0$. The new computation is

$$52 \text{ at } h \xrightarrow{0L} 176 \text{ at } h-1 \xrightarrow{0R} 185 \text{ at } h \xrightarrow{0R} 168 \text{ at } h+1.$$

Again the entire tape is unchanged and the endpoint is exactly the old one-step endpoint. The auxiliary rows 129/0 and 185/0 are unchanged rows of the literal candidate. The finite certificate checks them too.

Theorem 4.3 (Whole-tape CM215 equivalence)**.** *For every initial binary tape t and every head position h,*

$$\mathrm{Halts}(\mathrm{K215}, (0, h, t)) \iff \mathrm{Halts}(\mathrm{CM215}, (0, h, t)).$$

At corresponding comparison points the heads and complete tapes agree. If $z(n)$ of the first n old steps read zero in control 52 or 176, the corresponding new prefix has exactly $n + 2z(n)$ steps.

Proof. Use the relation just defined. Lemma 4.2 supplies its inductive invariant. The row identities and two local blocks establish positive advance with length one or three. The literal tables have matching terminal behavior under ϕ. Apply Theorem 3.1. The exact duration formula follows by adding the lengths of the local blocks. In particular, $n \le n + 2z(n) \le 3n$. □

The theorem quantifies arbitrary initial tapes *at control zero*. It does not assert that the invariant holds from an arbitrary control inside the modified fragment. For example, the comparison block for old control 52 reading zero uses the left-one clause essentially.

4.2 The concrete finite evidence

The independent Python checker enumerates all 1,710 admissible three-bit windows at the 215 working controls. Exactly four of these cases use three new steps; all other cases use one. The four count both values of the unused right neighbor in each of the two exceptional zero cases. Terminal behavior is checked separately. The window enumeration verifies the local hypotheses; the Lean induction extends them to arbitrary infinite tapes and arbitrarily long runs.

The formal table facts range over all 216 dense control values and both read bits, including the halt. Dense labels 52, 170, and 151 correspond to native controls 52, 176, and 156. Kernel computations are partitioned into small blocks; no `native_decide` or unproved assertion is used to accept the machine identities.

4.3 Connection to the existing endpoint

The concrete development uses function-valued tapes because the local invariant is particularly simple there. The repository's dense `Machine 215` represents a finite-support tape by a list of one-cell positions and uses `none` for the halt state. The bridge maps that tape to its read function and maps `none` to dense label 215. It proves that read-after-write and head movement commute with the embedding. The absorbed halt in the repository's total step is partialized before applying the positive-block theorem.

The old literal table is compared with the exact `KDense215.machine215`, not with a separately assumed equivalent table. A companion dense machine is defined from the new literal rows. This provides a direct blank-halting equivalence in the repository's machine semantics. The small integration wrapper composes that equivalence with the existing K215 ZFC theorem. The integration wrapper itself was not compiled in this session. The freshly checked results are the Core-machine equivalence and the parameterized `calibration_transfer` theorem. The complete 185-module calibration was not rebuilt in this packet.

At the ordinary internal level, if ZFC proves

$$\mathrm{Halts}(\mathrm{K215}, \mathrm{blank}) \leftrightarrow \mathrm{Halts}(\mathrm{CM215}, \mathrm{blank}) \quad \text{and} \quad \neg\,\mathrm{Halts}(\mathrm{K215}, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathrm{ZFC}),$$

then ZFC proves the requested CM215 equivalence by transitivity and negation. The local tape proof uses finite transition inspection and induction on natural-number execution lengths. It introduces no new set-theoretic axiom and no additional consistency premise.

5 Which 215-state candidate is easier to verify?

For adding a second verified endpoint, CM215 has the clearest advantage: its entire new burden is a local transport proof from a checked base. This conclusion is about proof reuse, not merely source size.

Candidate	Relevant design change	Consequence for a new proof
CM215	Six rows; same source; local one/three-step simulation	Reuse the complete K215 calibration, source proof, and startup. Best immediate choice for an additional endpoint.
Rebalanced R41	Nineteen native rows; 4,120 source addresses changed; maximum depth 47 to 41	Useful address-complexity reduction, but requires a new source correspondence and runtime bindings.
Commands120	DROP/NIP/MP command ordering	Reuses the same broad calculus but changes source/control and startup evidence.
Direct-transfer ST215	A direct 35-leaf transfer command and changed source association	Fewer primitive occurrences, but an additional command/compiler and protected-tail argument.
Zero-spare variants	Changed spare representation and startup seed	Additional representation and startup obligations; not automatically easier because the stored constants are smaller.

The original CM215 packet reports 6,176 runtime cases and 1,701,542 symbolic rules, slightly more than the primary machine's 1,698,790 rules. Thus it would be misleading to justify CM215 as easier merely by its full runtime-certificate size. Its advantage is that this second full runtime proof can be bypassed by the direct equivalence theorem. The primary K215 is still the established starting point for a complete calibration from first principles.[4, 1]

R41 is a sensible next subject if the aim is to simplify the decoder and source checker themselves. The retained packet reports 1,640,317 symbolic rules, about 3.44% fewer than the primary figure, and the same 95,714-step bootstrap. That arithmetic comparison does not demonstrate a faster kernel build.[6]

ST215 has 4,732 primitive occurrences and 6,151 guarded cases, with a 70,506-step blank prefix. Its command change means those smaller numbers must be weighed against a different semantic proof. The repository's intake notes distinguish delivered claims from checks rerun at intake; this report does not upgrade those notes into new independent formal verification.[7, 8, 9]

6 A concrete next simplification of the source proof

6.1 Remove unused offset generality carefully

The current K215 source proof compares the optimized path program with `DagPerm` using an offset segment-bisimulation certificate. Each entry records two checkpoint paths, an integer offset on register five, and two successor entries. All 2,240 offsets in this particular certificate are zero.[5]

The corresponding semantic relation is simply equality of register files. A specialized checker can avoid integer casts, offset propagation, and offset-zero side conditions. This should be implemented as an instance of a proved general interface, so future offset-using candidates can retain the existing checker.

There is a concrete trap: the existing zero-offset checker decodes shorter path codes than the offset checker. Our replay finds maximum checkpoint path lengths 45 and 176 on the two sides; the target DAG has height 178. The old zero-offset decoding bound is insufficient. Parameterize the decoder depth, or use a structurally terminating path representation, before switching this source proof to zero-offset checking. Dropping the offset field without preserving path decoding would change the theorem.

6.2 Separate local effects from addresses

Between register tests, a segment contains only increments and control movement. Its action on the register file is captured by an increment count for each affected register. A checkpoint can therefore carry:

$$(\text{tested register},\ \mathbf{v}_+,\ \mathbf{v}_0, \text{positive successor},\ \text{zero successor}),$$

or a halt label. The positive branch first performs the guarded decrement; the vectors describe increments in the following segment. This is the semantics already exploited by the repository's segment checker.

The supplied analyzer regenerates the existing certificate from the pinned input files and checks the effects on both sides. It finds only **32 distinct local effect signatures** among the 2,240 checkpoints. The signatures consist of the tested register and the two sparse increment vectors, with a separate halt signature.

These 32 signatures are not 32 complete proof states. Continuations matter. Starting with the signature partition and repeatedly refining by the two successor classes stabilizes at **1,423 classes**. The analyzer checks explicitly that every refinement refines the preceding partition and that the final partition preserves both local labels and successor classes. It writes the full quotient certificate.

Proposition 6.1 (Soundness of the checkpoint quotient)**.** *Suppose two checkpoint nodes have the same halt/test label, the same branch increment vectors, and successors in corresponding classes of a graph congruence. When started with the same register file, their checkpoint executions halt equivalently and preserve equality of the register files at corresponding checkpoints.*

Proof. If the label is halt, both terminate. Otherwise both test the same register value and hence select the same branch. The decrement, if performed, agrees; adding the same increment vector gives equal new register files. The successor nodes again belong to the same class. Induction on the number of checkpoint transitions transfers any finite halting execution in either direction. □

6.3 What this measurement does and does not buy

The graph quotient has 817 fewer classes than the original certificate, about 36.47% fewer. The effect library has only 32 distinct entries. A proof representation can exploit both facts, but these percentages are *not* measured reductions in Lean proof size or kernel time. Every literal path still needs a justified binding to its checkpoint node, including the root segment. A quotient without those bindings would prove a statement about a detached graph, not about the source.

The recommended implementation has three layers: a verified conversion from source paths to checkpoint effects, a compact graph-congruence certificate, and the existing halting interface. Cache shared effect proofs in named lemmas and store source addresses separately. Retain the current checker as a comparison implementation until the exact K215 endpoint and all assumption reports pass through the new route.

The source analyzer and quotient certificate are finite Python artifacts, not proof-assistant theorems. They supply a concrete target for that refactor. They also prevent confusing the 179-node compiled path-program DAG in the Lean source with the 202-subtree source representation used by the machine construction; those are different representations.

7 Completed searches for a 214-state machine

7.1 Pinned inputs and abstract contexts

The search suite contains fourteen literal 215-state tables: primary K215, CM215, commands120, composed, reencoded, zero-spare, zero-spare-z201, zero-spare-transfer, MP-first transfer, both rho6 orders, zero-spare MP-first, direct-transfer ST215, and rebalanced R41. Paths and hashes are recorded in `reduction_search/inputs.json`. This is a selected inventory, not a claim to enumerate every table or graph-isomorphism class in the repository.

An abstract configuration consists of a control and the fifteen bits at head offsets $-7, \ldots, 7$. Writes and shifts update known cells exactly; a newly exposed boundary bit may be either zero or one. Closing these transitions gives an inductive overapproximation of reachable windows. One initial domain uses every possible window at the start control; the other uses the zero window, which includes the actual blank tape but allows arbitrary distant exterior bits.

For changed rows, the filter seeks a positive common endpoint within 97 transitions on each side, without reading outside the known initial window. At a nonterminal join it requires equal heads, equal whole known windows, and related controls. Neither run has written outside the window, so this gives whole-tape equality. When both prefixes halt, terminal tape equality is unnecessary. A final move may leave the window; the next read may not occur outside it before a join.

7.2 Family A: inherited rows and one common branch

For every ordered pair (u, v) of distinct working controls, remove v and redirect references to v into u. All other rows retain their mapped actions. On each symbol the keeper may use either of the two mapped inherited actions. If exactly one branch agrees, the other branch is also allowed every write, direction, and remaining target, including the halt. There are 860 possible actions on such a free branch.

Every input has $215 \cdot 214 = 46,010$ ordered pairs. The total candidate specification count for one

initial domain is 8,854,464, and both domains therefore produce **17,708,928 full-specification evaluations**. No candidate survives. The count is of evaluated specifications; different specifications can describe the same literal machine.

Input	Specifications per domain
Primary / commands120 / composed / rebalanced	632,584 each
CM215 / zero-transfer / zero-MP-first	630,872 each
Reencoded	641,144
Zero-spare / z201	634,296 each
MP-first	629,160
rho6-120 / rho6-210	636,008 each
Direct transfer	620,600

The Python auditor independently recomputes these combinatorial counts from the tables; it does not merely accept the C++ search counters.

7.3 Family B: independently synthesized branches

For primary K215 and CM215, a second search permits both keeper actions to change. Each branch is synthesized independently from all 860 actions. Before rejoining, its new block may not read the opposite keeper branch. This restriction makes independent synthesis sound as a filter but excludes solutions that must alternate between the two new rows.

The search first tries zero-branch actions. If none passes for a pair, there is no need to enumerate its one-branch actions. Any pair with passing actions on both symbols would be sent to the full joint filter. No such complete pair was found.

Input	Initial domain	Individual branch evaluations
K215	All windows	39,883,360
K215	Zero window	39,883,360
CM215	All windows	39,871,320
CM215	Zero window	39,871,320
Total		159,509,360

This total is emphatically *not* a count of that many distinct 214-state machines. It counts individual branch specifications in the restricted synthesis procedure.

7.4 Controls and interpretation

Two small synthetic tables provide nonvacuous controls. A table with duplicate terminal roles produces ten Family-A survivors and 288 Family-B survivors; every survivor is freshly checked by an unchanged independent repository checker. A table with incompatible halting and forever-moving roles produces none. The packet retains all 32 large search receipts and a fresh selected replay. A separate root review recompiled the implementation, reran the controls, and reproduced the primary all-window Family-A record exactly apart from elapsed time.

Failure of a bounded join does not show that a candidate has the wrong blank behavior. The filter may encounter an abstract window that is unreachable in the concrete execution, need a larger window or longer join, or need a relation that changes the tape representation. Neither search family covers coordinated predecessor rewrites, arbitrary source recompilation, or every interacting two-row transformation. No 214-state impossibility theorem follows.

The practical lesson is to move beyond a single isolated merged control with all other mapped rows fixed. Coordinated incoming-row edits, source reassociation followed by allocation, or a tape marker that retains the former role are better targets for the next search. A candidate should carry the intended simulation relation from its construction, so its correctness can use the small transport interface.

8 Reproduction and integration

The archive is organized as a repository-ready research packet. The root `README.md` and `verify.py` describe exact commands and distinguish inexpensive finite audits from optional compiler checks and full search replay. The report source can be compiled with `pdflatex`; no external bibliography service is needed.

Path	Contents
`report/`	This article as editable LaTeX and compiled PDF.
`formal/`	Generic Lean/Coq transport, concrete CM215 Lean facts, invariant, simulation, and dense-machine bridge.
`machines/`	Exact K215 and CM215 dense tables and native references.
`source_analysis/`	Pinned source inputs, reference generator, structural analyzer, and graph quotient certificate.
`reduction_search/`	C++ search, fourteen exact inputs, independent controls, standard-library Python driver, and all bounded-search receipts.
`integration/`	Small endpoint wrapper for the existing ProveIt project and integration instructions.
`evidence/`	New successful compiler logs, assumption reports, fresh finite audits, and explicit scope records.

No compiled search executable is required or shipped. Kernel artifacts can be regenerated with the stated versions. The generic Coq check and the concrete Lean extension are separate checks. The original 185-module Lean and 237-source Rocq calibration records remain the baseline evidence; this packet does not relabel them as freshly rerun.

Integration should preserve the existing K215 endpoint and add CM215 through a short theorem composition. A successful complete project build should print assumptions for the resulting CM215 endpoint as well as for the new transport theorem. That final project-level audit will verify the imported calibration and new extension together.

9 Further research questions

1. **A verified local-certificate format.** Can the three-clause CM215 proof be generated from a small finite-window certificate whose soundness is shared by Lean and Rocq? A useful

first target is a format containing incoming-row predicates, whole-tape block equalities, and positive progress, with exact table hashes outside and literal rows inside the kernel boundary.

2. **Interacting keeper branches.** Can the second reduction search be extended to permit alternating between both synthesized rows, while solving their finite constraints jointly? This would remove a specific stated restriction, rather than merely increasing numerical search bounds.

3. **Coordinated marker rewrites.** Can the selector role reused by CM215 share an existing physical control after a few predecessor edits? The search relation may retain origin in a neighboring marker instead of requiring the same tape at every one-step boundary. The three-clause invariant suggests exactly which information must survive.

4. **A zero-offset source checker.** Parameterize decoding depth and prove that the current K215 certificate is a zero-offset instance. Measure generated source bytes, kernel object bytes, peak memory, and compile time separately. All four are relevant proof costs.

5. **A quotient-based source certificate.** Formalize the 32 effect templates and 1,423-class checkpoint quotient, including every literal-path binding and the root segment. Does the smaller semantic graph actually shorten the checked source proof, or do address proofs dominate the remaining cost?

6. **Proof-aware source optimization.** Search jointly for state count and new proof burden. A table requiring six local row facts may be a better formalization target than one with fewer primitive occurrences but a new scheduler invariant. A useful optimizer should report both a physical state count and a proposed reusable theorem interface.

7. **Arithmetic internalization.** Isolate the elementary machine transport and compiler layer in a weak arithmetic formalization, then connect it to the separately established internal consistency statement. Keep this milestone distinct from external Lean or Rocq proofs of the semantic equivalence.

The immediate deliverable is a reusable route to a second 215-state endpoint. A 214-state construction remains open after these particular searches, and the packet specifies which transformations and proof templates have actually been tested.

Attribution

K215, CM215, the numerical-calculus calibration, source/compiler architecture, original marker transformations, and existing verification framework are inherited from ProveIt and the upstream authors credited in its notices. The existing CM215 machine is not claimed as a new construction here. This packet contributes the compact invariant-based formal transport development, generic checked simulation lemmas, source-certificate analysis, and the explicitly scoped search extension. Copied source and table notices are retained. No global novelty claim is made for the general simulation or partition-refinement principles.

References

[1] ProveIt authors. *Formal correctness of the exact 215-state machine*. Pinned verification index, Lean/Rocq build and assumption records. Repository source.

[2] ProveIt authors. *Shared lemmas and one enumerator theorem*, with phases 1–3. Repository source.

[3] ProveIt authors. *KEndpoint215.lean*. Exact Lean endpoint.

[4] ProveIt authors and credited contributors. *CM215: conditional origin markers and ZFC consistency*. Machine, ordinary proof, and retained evidence.

[5] ProveIt authors. *KSource215Data.lean*, *RMBisim.lean*, and *RMOffsetBisim.lean*. Source certificate; Offset checker.

[6] ProveIt authors and credited contributors. *K215-R41: rebalanced source addresses*. Delivered packet; Intake status.

[7] ProveIt authors and credited contributors. *ST215: direct 35-leaf transfer through contextual reassociation*. Intake status.

[8] ProveIt authors and credited contributors. *A 215-state machine with DROP/NIP/MP command order*. Repository source.

[9] ProveIt authors and credited contributors. *Z215: zero-valued spare blocks and a two-truth startup*. Intake status.